LAB SHEET 2

1. MySQL RDBMS and MongoDB Document Model

```
-- MySQL (run in MySQL client)
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE Student (
  Student_ID INT PRIMARY KEY,
  Name VARCHAR(50),
  Course VARCHAR(30),
  Semester INT,
  Marks DECIMAL(5,2)
);

INSERT INTO Student VALUES
(1,'Aarav','BCA',3,82.5),
(2,'Diya','BCA',3,88.0),
(3,'Kabir','BBA',2,76.5),
(4,'Meera','BCA',5,91.0),
(5,'Rohan','BCom',4,79.0);

SELECT * FROM Student;

-- MongoDB (run in mongosh)
use CollegeDB
db.Students.insertMany([
  {Student_ID:1, Name:'Aarav', Course:'BCA', Semester:3, Marks:82.5},
  {Student_ID:2, Name:'Diya', Course:'BCA', Semester:3, Marks:88},
  {Student_ID:3, Name:'Kabir', Course:'BBA', Semester:2, Marks:76.5},
  {Student_ID:4, Name:'Meera', Course:'BCA', Semester:5, Marks:91},
  {Student_ID:5, Name:'Rohan', Course:'BCom', Semester:4, Marks:79}
])
db.Students.find()

-- Comparison:
-- RDBMS: fixed table schema, rows/columns, SQL, joins and strong relations.
-- MongoDB: flexible document schema, BSON documents, field-based queries,
-- nested data is convenient; horizontal scaling can use sharding.
```

2. MongoDB CollegeDB Student CRUD

```
use CollegeDB

db.Students.insertMany([
  {Student_ID:101, Name:'Aarav', Course:'BCA', Semester:3, Marks:82},
  {Student_ID:102, Name:'Diya', Course:'BCA', Semester:5, Marks:91},
  {Student_ID:103, Name:'Kabir', Course:'BBA', Semester:2, Marks:76},
  {Student_ID:104, Name:'Meera', Course:'BCom', Semester:4, Marks:85},
  {Student_ID:105, Name:'Rohan', Course:'BCA', Semester:1, Marks:79}
])

// 1. Display all
db.Students.find()

// 2. Find students in BCA
db.Students.find({Course:'BCA'})

// 3. Update marks
db.Students.updateOne({Student_ID:101}, {$set:{Marks:87}})

// 4. Delete one student
db.Students.deleteOne({Student_ID:105})

// 5. Count students
db.Students.countDocuments()
```

3. MongoDB Orders with Embedded Products

```
use CollegeDB

db.Orders.insertMany([
 {order_id:1, customer:{name:'Aarav', city:'Roorkee'},
  products:[{name:'Pen',price:10,qty:2},{name:'Book',price:80,qty:1}]},
 {order_id:2, customer:{name:'Diya', city:'Meerut'}, products:[{name:'Bag',price:700,qty:1}]},
 {order_id:3, customer:{name:'Kabir', city:'Delhi'}, products:[{name:'Book',price:80,qty:3}]},
 {order_id:4, customer:{name:'Meera', city:'Roorkee'}, products:[{name:'Bottle',price:150,qty:2}]},
 {order_id:5, customer:{name:'Rohan', city:'Delhi'},
  products:[{name:'Pen',price:10,qty:5},{name:'Bag',price:700,qty:1}]}
])

// 1. Orders of one customer
db.Orders.find({'customer.name':'Aarav'})

// 2. Update quantity of a product in order 1
db.Orders.updateOne(
 {order_id:1, 'products.name':'Pen'},
 {$set:{'products.$.qty':4}}
)

// 3. Calculate order total with aggregation
db.Orders.aggregate([
 {$unwind:'$products'},
 {$group:{_id:'$order_id', customer:{$first:'$customer.name'},
   total:{$sum:{$multiply:['$products.price','$products.qty']}}}},
 {$sort:{_id:1}}
])

// 4. Show orders whose calculated total is above 500
db.Orders.aggregate([
 {$unwind:'$products'},
 {$group:{_id:'$order_id', customer:{$first:'$customer'},
   total:{$sum:{$multiply:['$products.price','$products.qty']}}}},
 {$match:{total:{$gt:500}}}
])
```

4. Redis Key-Value Student Information System

```
# Run these commands in redis-cli
SET student:1 "Aarav"
SET student:2 "Diya"
SET student:3 "Kabir"
SET student:4 "Meera"
SET student:5 "Rohan"

# Retrieve a student by key
GET student:3

# Update a student's value
SET student:3 "Kabir Sharma"

# Delete a student
DEL student:5

# Display available student keys
KEYS student:*
```

5. Cassandra CollegeDB Students

```
-- Run in cqlsh
CREATE KEYSPACE IF NOT EXISTS CollegeDB
WITH replication = {'class':'SimpleStrategy','replication_factor':1};

USE CollegeDB;

CREATE TABLE IF NOT EXISTS Students (
  Student_ID int PRIMARY KEY,
  Name text,
  Course text,
  Semester int,
  Marks double
);

INSERT INTO Students (Student_ID,Name,Course,Semester,Marks) VALUES (1,'Aarav','BCA',3,82);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks) VALUES (2,'Diya','BCA',5,91);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks) VALUES (3,'Kabir','BBA',2,76);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks) VALUES (4,'Meera','BCom',4,85);
INSERT INTO Students (Student_ID,Name,Course,Semester,Marks) VALUES (5,'Rohan','BCA',1,79);

-- Display records
SELECT * FROM Students;

-- Cassandra requires query-friendly keys; create a table for course lookup
CREATE TABLE IF NOT EXISTS StudentsByCourse (
  Course text, Student_ID int, Name text, Semester int, Marks double,
  PRIMARY KEY (Course, Student_ID)
);
INSERT INTO StudentsByCourse (Course,Student_ID,Name,Semester,Marks)
VALUES ('BCA',1,'Aarav',3,82);
INSERT INTO StudentsByCourse (Course,Student_ID,Name,Semester,Marks)
VALUES ('BCA',2,'Diya',5,91);
SELECT * FROM StudentsByCourse WHERE Course='BCA';

UPDATE Students SET Marks=88 WHERE Student_ID=1;
DELETE FROM Students WHERE Student_ID=5;
```

6. HBase Students Table

```
# Run in HBase shell
create 'Students', 'personal', 'academic'

put 'Students', '1', 'personal:name', 'Aarav'
put 'Students', '1', 'personal:course', 'BCA'
put 'Students', '1', 'academic:semester', '3'
put 'Students', '1', 'academic:marks', '82'

put 'Students', '2', 'personal:name', 'Diya'
put 'Students', '2', 'personal:course', 'BCA'
put 'Students', '2', 'academic:semester', '5'
put 'Students', '2', 'academic:marks', '91'

put 'Students', '3', 'personal:name', 'Kabir'
put 'Students', '3', 'personal:course', 'BBA'
put 'Students', '3', 'academic:semester', '2'
put 'Students', '3', 'academic:marks', '76'

put 'Students', '4', 'personal:name', 'Meera'
put 'Students', '4', 'personal:course', 'BCom'
put 'Students', '4', 'academic:semester', '4'
put 'Students', '4', 'academic:marks', '85'

put 'Students', '5', 'personal:name', 'Rohan'
put 'Students', '5', 'personal:course', 'BCA'
put 'Students', '5', 'academic:semester', '1'
put 'Students', '5', 'academic:marks', '79'

# Retrieve one student's details
get 'Students', '1'

# Update information
put 'Students', '1', 'academic:marks', '88'

# Display all records
scan 'Students'

# Delete one record
deleteall 'Students', '5'
```

7. Neo4j Student–Course Graph

```
// Run in Neo4j Browser
CREATE
 (s1:Student {id:1,name:'Aarav'}),
 (s2:Student {id:2,name:'Diya'}),
 (s3:Student {id:3,name:'Kabir'}),
 (s4:Student {id:4,name:'Meera'}),
 (s5:Student {id:5,name:'Rohan'}),
 (c1:Course {name:'BCA'}),
 (c2:Course {name:'BBA'}),
 (c3:Course {name:'BCom'}),
 (s1)-[:ENROLLED_IN]->(c1),
 (s2)-[:ENROLLED_IN]->(c1),
 (s3)-[:ENROLLED_IN]->(c2),
 (s4)-[:ENROLLED_IN]->(c3),
 (s5)-[:ENROLLED_IN]->(c1);

// Display all students
MATCH (s:Student) RETURN s;

// Find students enrolled in BCA
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {name:'BCA'})
RETURN s.name;

// Courses taken by a particular student
MATCH (:Student {name:'Aarav'})-[:ENROLLED_IN]->(c:Course)
RETURN c.name;

// Add a relationship
MATCH (s:Student {name:'Kabir'}), (c:Course {name:'BCA'})
CREATE (s)-[:ENROLLED_IN]->(c);

// Remove that relationship
MATCH (:Student {name:'Kabir'})-[r:ENROLLED_IN]->(:Course {name:'BCA'})
DELETE r;
```

8. Implement Dataset in MongoDB, Cassandra and Neo4j

```
// MongoDB (mongosh)
use CollegeDB
db.StudentDemo.insertMany([
 {id:1,name:'Aarav',course:'BCA',marks:82},
 {id:2,name:'Diya',course:'BCA',marks:91},
 {id:3,name:'Kabir',course:'BBA',marks:76},
 {id:4,name:'Meera',course:'BCom',marks:85},
 {id:5,name:'Rohan',course:'BCA',marks:79}
])
db.StudentDemo.find({course:'BCA'})

// Cassandra (cqlsh; keyspace CollegeDB must exist)
USE CollegeDB;
CREATE TABLE IF NOT EXISTS StudentDemo (
 id int PRIMARY KEY, name text, course text, marks double
);
INSERT INTO StudentDemo (id,name,course,marks) VALUES (1,'Aarav','BCA',82);
INSERT INTO StudentDemo (id,name,course,marks) VALUES (2,'Diya','BCA',91);
INSERT INTO StudentDemo (id,name,course,marks) VALUES (3,'Kabir','BBA',76);
SELECT * FROM StudentDemo;

// Neo4j Browser
CREATE (:Student {id:1,name:'Aarav',course:'BCA',marks:82}),
       (:Student {id:2,name:'Diya',course:'BCA',marks:91}),
       (:Student {id:3,name:'Kabir',course:'BBA',marks:76});
MATCH (s:Student) WHERE s.course='BCA' RETURN s;

// Comparison:
// MongoDB: document/BSON; queries use find/aggregation; flexible schema.
// Cassandra: wide-column; CQL; designed for high-volume distributed writes.
// Neo4j: graph nodes/relationships; Cypher; suited to connected data.
// All can scale, but the best choice depends on query patterns and workload.
```

9. Deploy MongoDB Locally and Perform CRUD

```
# Start MongoDB service (method varies by OS/install)
# Then open a terminal and run: mongosh

use LocalLabDB
db.createCollection('Students')

db.Students.insertMany([
 {id:1,name:'Aarav',course:'BCA'},
 {id:2,name:'Diya',course:'BCA'},
 {id:3,name:'Kabir',course:'BBA'}
])

// Create/read/update/delete
db.Students.find()
db.Students.find({course:'BCA'})
db.Students.updateOne({id:1}, {$set:{course:'MCA'}})
db.Students.deleteOne({id:3})
db.Students.find()

// Verification: the final find should show the remaining updated records.
// Ensure the MongoDB server is running before opening mongosh.
```

10. Demonstrate MongoDB Sharding (Lab Setup)

```
// This is a configuration outline for a sharded MongoDB test cluster.
// A real cluster needs config servers, mongos, and shard replica sets.
// Run cluster commands in mongosh connected to mongos, not a standalone server.

// 1. Add shards (replace host:port with your configured replica-set endpoints)
sh.addShard('shard1RS/localhost:27018')
sh.addShard('shard2RS/localhost:27019')

// 2. Enable sharding for database and collection
sh.enableSharding('CollegeDB')
sh.shardCollection('CollegeDB.ShardedStudents', {Student_ID: 'hashed'})

// 3. Insert at least 20 records
const docs = [];
for (let i = 1; i <= 20; i++) {
  docs.push({Student_ID:i, Name:'Student'+i, Course:i%2?'BCA':'BBA', Marks:60+i});
}
db.getSiblingDB('CollegeDB').ShardedStudents.insertMany(docs)

// 4. Verify records and sharding status
db.getSiblingDB('CollegeDB').ShardedStudents.find()
sh.status()

// Benefit: distributes data and query/load across shards.
// Note: the addShard commands only work after the replica sets and mongos
// have been configured and started; they do not create servers automatically.
```

11. MongoDB Primary–Secondary Replication

```
// Run on a test installation with three separate mongod instances.
// Start each instance with a distinct dbpath/port and the same replica-set name:
// mongod --replSet rsLab --port 27017 --dbpath ./data1 --bind_ip localhost
// mongod --replSet rsLab --port 27018 --dbpath ./data2 --bind_ip localhost
// mongod --replSet rsLab --port 27019 --dbpath ./data3 --bind_ip localhost
// Use separate directories for each server; create directories first.

// Connect to one instance with mongosh, then:
rs.initiate({
 _id:'rsLab',
 members:[
  {_id:0,host:'localhost:27017'},
  {_id:1,host:'localhost:27018'},
  {_id:2,host:'localhost:27019'}
 ]
})

rs.status()

// Insert data on the PRIMARY
use CollegeDB
db.ReplicationDemo.insertOne({id:1,name:'Aarav',course:'BCA'})

// Read from a secondary in another mongosh session
// Connect to localhost:27018, then:
rs.secondaryOk()
db.getSiblingDB('CollegeDB').ReplicationDemo.find()

// If primary fails, an eligible secondary may be elected primary.
// Replication improves availability but requires multiple running instances.
```

12. Cassandra Peer-to-Peer Replication

```
-- Configure multiple Cassandra nodes with the same cluster name and
-- appropriate seed/contact-point settings in cassandra.yaml, then start them.
-- In cqlsh:
CREATE KEYSPACE IF NOT EXISTS CollegeP2P
WITH replication = {'class':'NetworkTopologyStrategy','datacenter1':3};

USE CollegeP2P;

CREATE TABLE IF NOT EXISTS Students (
 Student_ID int PRIMARY KEY, Name text, Course text, Marks double
);

INSERT INTO Students (Student_ID,Name,Course,Marks) VALUES (1,'Aarav','BCA',82);
INSERT INTO Students (Student_ID,Name,Course,Marks) VALUES (2,'Diya','BCA',91);
INSERT INTO Students (Student_ID,Name,Course,Marks) VALUES (3,'Kabir','BBA',76);

SELECT * FROM Students;
-- Verify node membership/replicas with nodetool status and nodetool describecluster.
-- A replication factor of 3 requires at least 3 suitable nodes for full replication.
-- Cassandra nodes are peers; there is no fixed master for all writes.
```

13. Sharding + Replication Architecture (MongoDB)

```
// Architecture outline:
// Application -> mongos router -> Shard 1 replica set (primary + secondary)
//                            -> Shard 2 replica set (primary + secondary)
// Each shard stores a subset of the collection and has replicas for availability.

// After configuring config servers, mongos, and both shard replica sets:
sh.addShard('shard1RS/localhost:27018,localhost:27028')
sh.addShard('shard2RS/localhost:27019,localhost:27029')
sh.enableSharding('CollegeDB')
sh.shardCollection('CollegeDB.StudentScale', {Student_ID:'hashed'})

const rows = [];
for (let i=1; i<=30; i++) {
  rows.push({Student_ID:i, Name:'Student'+i, Course:i%3===0?'BBA':'BCA', Marks:50+i});
}
db.getSiblingDB('CollegeDB').StudentScale.insertMany(rows)

sh.status()
db.getSiblingDB('CollegeDB').StudentScale.countDocuments()

// Sharding distributes data for scalability; replica sets maintain copies
// and can elect a new primary after a failure. Actual hostnames/ports must
// match the cluster you have configured.
```

14. MongoDB MapReduce: Student Count and Average Marks

```
use CollegeDB

db.CourseStats.drop()
db.StudentsForMR.insertMany([
 {Student_ID:1,Name:'Aarav',Course:'BCA',Marks:82},
 {Student_ID:2,Name:'Diya',Course:'BCA',Marks:90},
 {Student_ID:3,Name:'Kabir',Course:'BBA',Marks:76},
 {Student_ID:4,Name:'Meera',Course:'BBA',Marks:84},
 {Student_ID:5,Name:'Rohan',Course:'BCA',Marks:79}
])

// MapReduce is deprecated/removed in some newer MongoDB versions.
// This classic example works only on versions that still support mapReduce.
var mapFunction = function () {
  emit(this.Course, {count:1, total:this.Marks});
};
var reduceFunction = function (key, values) {
  var result = {count:0, total:0};
  values.forEach(function (v) {
    result.count += v.count;
    result.total += v.total;
  });
  return result;
};

db.StudentsForMR.mapReduce(
  mapFunction,
  reduceFunction,
  {out:'CourseStats'}
);

db.CourseStats.find().forEach(function (doc) {
  print(doc._id + ' count=' + doc.value.count +
        ' average=' + (doc.value.total / doc.value.count));
});

// Modern alternative (supported by current MongoDB versions):
db.StudentsForMR.aggregate([
 {$group:{_id:'$Course', count:{$sum:1}, averageMarks:{$avg:'$Marks'}}}
])
```

15. Database Distribution Models: Summary and Diagram

```
// A. Single Server
// Client -> One database server
// Data stays on one server; failure can make the database unavailable.
// Advantage: simple setup. Limitation: limited scale/single point of failure.
// Use: small college application.

// B. Sharding
// Client -> Router -> Shard 1 | Shard 2 | Shard 3
// Different data partitions live on different shards.
// Advantage: horizontal scale. Limitation: more operational complexity.
// Use: large catalog or high-volume dataset.

// C. Primary–Secondary Replication
// Client -> Primary -> Secondary 1 / Secondary 2
// Primary accepts writes and copies changes to secondaries.
// Advantage: failover/read scaling. Limitation: replication lag and election rules.
// Use: application requiring availability.

// D. Peer-to-Peer Replication (e.g., Cassandra)
// Client -> Node 1 <-> Node 2 <-> Node 3
// Nodes are peers and data is replicated according to replication factor.
// Advantage: distributed availability. Limitation: consistency tuning is important.
// Use: globally distributed, high-write workloads.

// E. Sharding + Replication
// Router -> Shard 1 (Primary + Replicas)
//        -> Shard 2 (Primary + Replicas)
// Data is partitioned between shards and replicated within each shard.
// Advantage: scalability plus fault tolerance.
// Limitation: setup, monitoring and recovery are more complex.
// Use: large-scale applications with high availability.
```